

Documentación Completa del Modelo de Evaluación Inteligente

Introducción para Principiantes

Este documento explica detalladamente el código de un sistema avanzado de evaluación automática de contenido. Si eres principiante, no te preocupes - vamos a descomponer cada parte del código y explicar no sólo QUÉ hace, sino CÓMO y POR QUÉ lo hace, como si estuviéramos construyéndolo juntos desde cero.

Índice

1. [Visión General del Sistema](#1-visión-general-del-sistema)
2. [Importaciones y Configuración Inicial](#2-importaciones-y-configuración-inicial)
3. [Preparación de Datos: TextDataset](#3-preparación-de-datos-textdataset)
4. [Modelos de Redes Neuronales](#4-modelos-de-redes-neuronales)
5. [Clase Principal:
EnhancedModeloEvaluacionInteligente](#5-clase-principal-enhancedmodeloevaluacioninteligente)
6. [Procesamiento de Texto y Vectorización](#6-procesamiento-de-texto-y-vectorización)
7. [Análisis de Métricas](#7-análisis-de-métricas)
8. [Generación de Retroalimentación](#8-generación-de-retroalimentación)
9. [Entrenamiento del Modelo](#9-entrenamiento-del-modelo)
10. [Aplicación Práctica](#10-aplicación-práctica)
11. [Conclusiones](#11-conclusiones)

1. Visión General del Sistema

¿Qué hace este sistema?

Este sistema es como un "profesor automático" que puede evaluar dos tipos principales de contenido:

- **Contenido textual:** ensayos, informes, respuestas a preguntas
- **Código de programación:** soluciones a problemas, implementaciones de algoritmos

El sistema no solo asigna una calificación numérica, sino que también proporciona retroalimentación detallada, identifica fortalezas y debilidades, y recomienda recursos educativos personalizados.

¿Cómo funciona a grandes rasgos?

1. **Recibe el contenido** a evaluar (texto o código)
2. **Detecta automáticamente** si es texto o código
3. **Analiza múltiples aspectos** (claridad, estructura, funcionalidad, etc.)
4. **Utiliza redes neuronales** para predecir una calificación
5. **Genera retroalimentación personalizada** según el estilo de aprendizaje del estudiante

¿Por qué es importante?

Este sistema permite:

- Evaluación consistente y objetiva
- Retroalimentación inmediata y detallada
- Personalización según las necesidades del estudiante
- Escalabilidad para evaluar grandes cantidades de trabajos

2. Importaciones y Configuración Inicial

Vamos a analizar las primeras líneas del código:

```
```python
import numpy as np
import re
import torch
import torch.nn as nn
from torch.utils.data import Dataset, DataLoader
import torch.optim as optim
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity
from nltk.corpus import stopwords
import json
import os
import pickle
import nltk
import logging
import random
from collections import defaultdict
from typing import Dict, List, Tuple, Optional, Union, Any
import mysql.connector
from mysql.connector import Error
```
```

¿Por qué necesitamos estas bibliotecas?

- **numpy (np)**: Para operaciones matemáticas y manipulación de arrays. Es como la "calculadora avanzada" de Python.
- **re**: Para expresiones regulares, que nos permiten buscar y manipular patrones en texto.
- **torch**: Es PyTorch, el framework de aprendizaje profundo que usaremos para crear y entrenar nuestras redes neuronales.
- **torch.nn**: Contiene los componentes para construir redes neuronales (capas, funciones de activación, etc.).

- **Dataset, DataLoader**: Clases de PyTorch para manejar conjuntos de datos y cargarlos eficientemente durante el entrenamiento.
- **optim**: Contiene optimizadores como Adam que ajustan los parámetros de la red durante el entrenamiento.
- **TfidfVectorizer**: Convierte texto en vectores numéricos usando el método TF-IDF (explicaremos esto más adelante).
- **cosine_similarity**: Calcula la similitud entre vectores, útil para comparar textos.
- **stopwords**: Palabras comunes (como "el", "la", "y") que generalmente se ignoran en el procesamiento de texto.
- **json, os, pickle**: Para operaciones de archivo y serialización de objetos.
- **nltk**: Natural Language Toolkit, para procesamiento de lenguaje natural.
- **logging**: Para registrar información durante la ejecución del programa.
- **random**: Para operaciones que requieren aleatoriedad.
- **defaultdict**: Un diccionario especial que proporciona valores predeterminados para claves inexistentes.
- **typing**: Para anotaciones de tipo, que hacen el código más legible y ayudan a detectar errores.
- **mysql.connector**: Para conectarse a bases de datos MySQL.

Configuración de Logging

```
```python
Configurar logging
logging.basicConfig(level=logging.INFO, format='%(asctime)s - %(name)s - %(levelname)s - %(message)s')
logger = logging.getLogger(__name__)
```
```

¿Qué es el logging y por qué es importante?

El logging (registro) es como tener un "diario" de lo que hace tu programa. En lugar de usar `print()` para depurar, usamos logging porque:

1. **Podemos controlar el nivel de detalle**: INFO para información general, WARNING para advertencias, ERROR para errores, etc.
2. **Incluye automáticamente información útil**: hora, nombre del módulo, nivel
3. **Podemos dirigir los logs a diferentes destinos**: consola, archivos, servicios externos

En este caso, configuramos el logging para mostrar mensajes de nivel INFO o superior, con un formato que incluye la hora, el nombre del módulo, el nivel y el mensaje.

Descarga de Recursos NLTK

```
```python
Descargar recursos de NLTK necesarios si no existen
try:
```

```

 nltk.data.find('corpora/stopwords')
 nltk.data.find('tokenizers/punkt')
 nltk.data.find('corpora/wordnet')
except LookupError:
 nltk.download('stopwords')
 nltk.download('punkt')
 nltk.download('wordnet')
'''

```

### ¿Qué son estos recursos y por qué los necesitamos?

- **stopwords**: Palabras comunes que generalmente se ignoran en el análisis de texto (como "el", "la", "y").
- **punkt**: Un tokenizador que divide el texto en oraciones.
- **wordnet**: Una base de datos léxica que agrupa palabras en conjuntos de sinónimos.

Este código primero intenta encontrar estos recursos localmente. Si no existen, los descarga. Es como verificar si tienes las herramientas necesarias antes de empezar a trabajar, y si no las tienes, las consigues.

---

## 3. Preparación de Datos: TextDataset

```

'''python
class TextDataset(Dataset):
 def __init__(self, entradas, salidas, tokenizer):
 self.entradas = entradas
 self.salidas = salidas
 self.tokenizer = tokenizer

 def __len__(self):
 return len(self.entradas)

 def __getitem__(self, idx):
 # Vectorize the input text
 entrada = self.tokenizer.transform([self.entradas[idx]]).toarray()[0]

 # Ensure consistent vector size
 if len(entrada) < 5000:
 entrada = np.pad(entrada, (0, 5000 - len(entrada)), 'constant')
 elif len(entrada) > 5000:
 entrada = entrada[:5000]

 salida = float(self.salidas[idx])
 return torch.tensor(entrada, dtype=torch.float32), torch.tensor(salida,
dtype=torch.float32)
'''

```

...

### ### ¿Qué es un Dataset en PyTorch y por qué lo necesitamos?

Un Dataset en PyTorch es como una colección especializada que prepara nuestros datos para el entrenamiento. Necesitamos crear nuestra propia clase porque nuestros datos (texto) requieren un procesamiento especial antes de poder usarse en una red neuronal.

### ### Métodos de la clase TextDataset:

- **\*\*\*\*init\*\*\*\***: El constructor que inicializa el dataset con:
  - **\*\*entradas\*\***: Los textos a evaluar
  - **\*\*salidas\*\***: Las calificaciones correspondientes
  - **\*\*tokenizer\*\***: El vectorizador que convertirá texto en números
  
- **\*\*\*\*len\*\*\*\***: Devuelve el número de ejemplos en el dataset. PyTorch necesita saber cuántos ejemplos hay.
- **\*\*\*\*getitem\*\*\*\***: Devuelve un ejemplo específico por su índice. Este método es crucial porque:
  1. Convierte el texto en un vector numérico usando el tokenizer
  2. Asegura que todos los vectores tengan el mismo tamaño (5000 elementos)
    1. Si es más pequeño, lo rellena con ceros
    2. Si es más grande, lo trunca
  
- 3. Convierte el vector y la calificación en tensores de PyTorch

### ### ¿Por qué necesitamos vectores del mismo tamaño?

Las redes neuronales requieren entradas de dimensiones consistentes. Es como si todos los estudiantes tuvieran que usar el mismo formato de papel para sus exámenes - facilita el procesamiento.

---

## ## 4. Modelos de Redes Neuronales

### ### EnhancedContenidoEvaluator

```

```python
class EnhancedContenidoEvaluador(nn.Module):
    """Modelo mejorado para evaluar diferentes tipos de contenido con arquitectura más profunda"""
    def __init__(self, input_dim=5000, hidden_dim=768, output_dim=1):
        super(EnhancedContenidoEvaluador, self).__init__()
        # Arquitectura más profunda con residual connections
        self.fc1 = nn.Linear(input_dim, hidden_dim)
        self.bn1 = nn.BatchNorm1d(hidden_dim)
        self.dropout1 = nn.Dropout(0.3)

        # Bloque residual 1
        self.res1_fc1 = nn.Linear(hidden_dim, hidden_dim)
        self.res1_bn1 = nn.BatchNorm1d(hidden_dim)
        self.res1_dropout = nn.Dropout(0.3)
        self.res1_fc2 = nn.Linear(hidden_dim, hidden_dim)
        self.res1_bn2 = nn.BatchNorm1d(hidden_dim)

        # Bloque residual 2
        self.res2_fc1 = nn.Linear(hidden_dim, hidden_dim)
        self.res2_bn1 = nn.BatchNorm1d(hidden_dim)
        self.res2_dropout = nn.Dropout(0.3)
        self.res2_fc2 = nn.Linear(hidden_dim, hidden_dim)
        self.res2_bn2 = nn.BatchNorm1d(hidden_dim)

        # Capa de reducción
        self.fc_reduce = nn.Linear(hidden_dim, hidden_dim // 2)
        self.bn_reduce = nn.BatchNorm1d(hidden_dim // 2)
        self.dropout_reduce = nn.Dropout(0.2)

        # Múltiples cabezas para diferentes criterios de evaluación
        self.head_relevancia = nn.Linear(hidden_dim // 2, 1)
        self.head_profundidad = nn.Linear(hidden_dim // 2, 1)
        self.head_claridad = nn.Linear(hidden_dim // 2, 1)
        self.head_originalidad = nn.Linear(hidden_dim // 2, 1)
        self.head_global = nn.Linear(hidden_dim // 2, 1)

        self.relu = nn.ReLU()
        self.sigmoid = nn.Sigmoid()

    def forward(self, x):
        # Capa inicial
        x = self.fc1(x)
        x = self.bn1(x)
        x = self.relu(x)
        x = self.dropout1(x)

```

```

# Bloque residual 1
residual = x
out = self.res1_fc1(x)
out = self.res1_bn1(out)
out = self.relu(out)
out = self.res1_dropout(out)
out = self.res1_fc2(out)
out = self.res1_bn2(out)
out += residual # Conexión residual
out = self.relu(out)

# Bloque residual 2
residual = out
out = self.res2_fc1(out)
out = self.res2_bn1(out)
out = self.relu(out)
out = self.res2_dropout(out)
out = self.res2_fc2(out)
out = self.res2_bn2(out)
out += residual # Conexión residual
out = self.relu(out)

# Reducción de dimensionalidad
x = self.fc_reduce(out)
x = self.bn_reduce(x)
x = self.relu(x)
x = self.dropout_reduce(x)

# Salidas para cada criterio (escaladas a 0-10)
relevancia = self.sigmoid(self.head_relevancia(x)) * 10
profundidad = self.sigmoid(self.head_profundidad(x)) * 10
claridad = self.sigmoid(self.head_claridad(x)) * 10
originalidad = self.sigmoid(self.head_originalidad(x)) * 10

# Calificación global
global_score = self.sigmoid(self.head_global(x)) * 10

return {
    'relevancia': relevancia,
    'profundidad': profundidad,
    'claridad': claridad,
    'originalidad': originalidad,
    'global': global_score
}
...

```

¿Qué es una red neuronal y cómo funciona?

Una red neuronal es un modelo de aprendizaje automático inspirado en el cerebro humano. Consiste en capas de "neuronas" conectadas entre sí. Cada neurona recibe entradas, las procesa y produce una salida.

Componentes clave de esta red neuronal:

1. **Capas lineales (nn.Linear)**: Realizan transformaciones lineales de los datos. Son como ecuaciones de la forma $y = wx + b$, donde w son pesos y b es un sesgo.
2. **Normalización por lotes (nn.BatchNorm1d)**: Normaliza las activaciones para mejorar la estabilidad y velocidad del entrenamiento. Es como asegurarse de que todos los datos están en una escala similar.
3. **Dropout (nn.Dropout)**: Desactiva aleatoriamente algunas neuronas durante el entrenamiento para prevenir el sobreajuste. Es como obligar a la red a no depender demasiado de ninguna neurona específica.
4. **Función de activación ReLU**: Introduce no-linealidad en la red. Simplemente convierte valores negativos en cero y deja los positivos sin cambios.
5. **Función sigmoid**: Convierte cualquier número en un valor entre 0 y 1. Aquí se usa para escalar las salidas al rango 0-10 (multiplicando por 10).

Arquitectura especial: Bloques Residuales

Los bloques residuales son una innovación clave que permite entrenar redes más profundas. Funcionan así:

1. Toman una entrada y la procesan a través de varias capas
2. Luego suman la entrada original a la salida procesada
3. Esto crea un "atajo" para que la información fluya más fácilmente

Es como si además de pasar por un camino largo y complejo, la información también pudiera tomar un atajo directo.

Múltiples cabezas de evaluación

Esta red tiene cinco "cabezas" o salidas diferentes:

- **Relevancia**: ¿El contenido aborda el tema solicitado?
- **Profundidad**: ¿El análisis es superficial o profundo?
- **Claridad**: ¿Las ideas están expresadas claramente?
- **Originalidad**: ¿El contenido es original o repetitivo?
- **Global**: Calificación general

Cada cabeza se especializa en evaluar un aspecto específico del contenido, similar a cómo un profesor evalúa diferentes criterios.

EnhancedCodigoEvaluator

```

python
class EnhancedCodigoEvaluador(nn.Module):
    """Modelo especializado mejorado para evaluar código de programación"""
    def __init__(self, input_dim=5000, hidden_dim=768, output_dim=1):
        super(EnhancedCodigoEvaluador, self).__init__()
        # Arquitectura más profunda con residual connections
        self.fc1 = nn.Linear(input_dim, hidden_dim)
        self.bn1 = nn.BatchNorm1d(hidden_dim)
        self.dropout1 = nn.Dropout(0.3)

        # Bloque residual 1
        self.res1_fc1 = nn.Linear(hidden_dim, hidden_dim)
        self.res1_bn1 = nn.BatchNorm1d(hidden_dim)
        self.res1_dropout = nn.Dropout(0.3)
        self.res1_fc2 = nn.Linear(hidden_dim, hidden_dim)
        self.res1_bn2 = nn.BatchNorm1d(hidden_dim)

        # Bloque residual 2
        self.res2_fc1 = nn.Linear(hidden_dim, hidden_dim)
        self.res2_bn1 = nn.BatchNorm1d(hidden_dim)
        self.res2_dropout = nn.Dropout(0.3)
        self.res2_fc2 = nn.Linear(hidden_dim, hidden_dim)
        self.res2_bn2 = nn.BatchNorm1d(hidden_dim)

        # Capa de reducción
        self.fc_reduce = nn.Linear(hidden_dim, hidden_dim // 2)
        self.bn_reduce = nn.BatchNorm1d(hidden_dim // 2)
        self.dropout_reduce = nn.Dropout(0.2)

        # Cabezas específicas para evaluación de código
        self.head_funcionalidad = nn.Linear(hidden_dim // 2, 1)
        self.head_eficiencia = nn.Linear(hidden_dim // 2, 1)
        self.head_estilo = nn.Linear(hidden_dim // 2, 1)
        self.head_documentacion = nn.Linear(hidden_dim // 2, 1)
        self.head_global = nn.Linear(hidden_dim // 2, 1)

        self.relu = nn.ReLU()
        self.sigmoid = nn.Sigmoid()

    def forward(self, x):
        # Capa inicial
        x = self.fc1(x)
        x = self.bn1(x)
        x = self.relu(x)
        x = self.dropout1(x)

        # Bloque residual 1

```

```

    residual = x
    out = self.res1_fc1(x)
    out = self.res1_bn1(out)
    out = self.relu(out)
    out = self.res1_dropout(out)
    out = self.res1_fc2(out)
    out = self.res1_bn2(out)
    out += residual # Conexión residual
    out = self.relu(out)

    # Bloque residual 2
    residual = out
    out = self.res2_fc1(out)
    out = self.res2_bn1(out)
    out = self.relu(out)
    out = self.res2_dropout(out)
    out = self.res2_fc2(out)
    out = self.res2_bn2(out)
    out += residual # Conexión residual
    out = self.relu(out)

    # Reducción de dimensionalidad
    x = self.fc_reduce(out)
    x = self.bn_reduce(x)
    x = self.relu(x)
    x = self.dropout_reduce(x)

    # Salidas para cada criterio (escaladas a 0-10)
    funcionalidad = self.sigmoid(self.head_funcionalidad(x)) * 10
    eficiencia = self.sigmoid(self.head_eficiencia(x)) * 10
    estilo = self.sigmoid(self.head_estilo(x)) * 10
    documentacion = self.sigmoid(self.head_documentacion(x)) * 10

    # Calificación global
    global_score = self.sigmoid(self.head_global(x)) * 10

    return {
        'funcionalidad': funcionalidad,
        'eficiencia': eficiencia,
        'estilo': estilo,
        'documentacion': documentacion,
        'global': global_score
    }
...

```

Este modelo es muy similar al anterior en su arquitectura, pero está especializado para evaluar código. Las principales diferencias están en las cabezas de evaluación:

- ****Funcionalidad****: ¿El código funciona correctamente?
- ****Eficiencia****: ¿El código es eficiente en tiempo/recursos?
- ****Estilo****: ¿El código sigue buenas prácticas de estilo?
- ****Documentación****: ¿El código está bien documentado?
- ****Global****: Calificación general

DummyModel

```
```python
class DummyModel:
 """Modelo dummy que siempre devuelve un valor predeterminado."""
 def __init__(self):
 self.logger = logging.getLogger(__name__)
 self.logger.info("Iniciando modelo dummy")

 def eval(self):
 """Método para compatibilidad con PyTorch."""
 return self

 def __call__(self, *args, **kwargs):
 """Devuelve un valor predeterminado."""
 return torch.tensor([[0.75]], dtype=torch.float32) # Valor de dificultad media

 def to(self, device):
 """Método para compatibilidad con PyTorch."""
 return self
```
```

¿Qué es un modelo dummy y por qué lo necesitamos?

Un modelo dummy es un modelo simplificado que siempre devuelve el mismo valor, independientemente de la entrada. Es como un "plan B" que usamos cuando:

1. Los modelos principales fallan por alguna razón
2. Estamos probando el sistema y no necesitamos predicciones reales
3. No tenemos suficientes datos para entrenar un modelo completo

Este modelo dummy:

- Siempre devuelve 0.75 (en una escala de 0 a 1, que luego se escala a 0-10)
- Implementa métodos como `eval()` y `to()` para ser compatible con la interfaz de PyTorch
- Registra su inicialización para seguimiento

5. Clase Principal: EnhancedModeloEvaluacionInteligente

Esta es la clase principal que integra todos los componentes del sistema. Vamos a analizar su inicialización:

```
```python
class EnhancedModeloEvaluacionInteligente:
 """
 Modelo integrado mejorado para evaluación inteligente de contenido.
 Detecta automáticamente el tipo de contenido y aplica el evaluador adecuado.
 """

 def __init__(self, db_config=None, use_sbert=False):
 """
 Inicializa el modelo de evaluación inteligente mejorado.

 Args:
 db_config: Configuración para la conexión a la base de datos
 use_sbert: Si es True, utiliza modelos SBERT para análisis semántico profundo
 """

 # Configurar stopwords en español
 self.spanish_stopwords = stopwords.words('spanish')

 # Vectorizador TF-IDF mejorado
 self.vectorizer = TfidfVectorizer(
 stop_words=self.spanish_stopwords,
 max_features=5000,
 ngram_range=(1, 3), # Incluye trigramas para capturar frases más complejas
 min_df=2, # Ignora términos que aparecen en menos de 2 documentos
 max_df=0.95 # Ignora términos que aparecen en más de 95% de los
documents
)

 # Configurar dispositivo para PyTorch
 self.device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
 logger.info(f"Usando dispositivo: {self.device}")

 # Inicializar modelos especializados mejorados
 try:
 # Crear modelos
 self.modelo_contenido = EnhancedContenidoEvaluator(input_dim=5000,
hidden_dim=768)
 self.modelo_codigo = EnhancedCodigoEvaluator(input_dim=5000, hidden_dim=768)

 # Mover modelos al dispositivo
 self.modelo_contenido = self.modelo_contenido.to(self.device)
 self.modelo_codigo = self.modelo_codigo.to(self.device)

```

```

 # Optimizadores
 self.optimizer_contenido = optim.Adam(self.modelo_contenido.parameters(),
lr=0.001, weight_decay=1e-5)
 self.optimizer_codigo = optim.Adam(self.modelo_codigo.parameters(), lr=0.001,
weight_decay=1e-5)

 # Función de pérdida
 self.criterion = nn.MSELoss()

 except Exception as e:
 logger.error(f"Error al inicializar modelos: {e}")
 self.modelo_contenido = DummyModel()
 self.modelo_codigo = DummyModel()

 # Modelo SBERT para análisis semántico profundo (opcional)
 self.use_sbert = use_sbert
 if use_sbert:
 try:
 # Intentar importar sentence-transformers
 from sentence_transformers import SentenceTransformer
 # Cargar modelo de sentence-transformers para español
 self.sbert_model = SentenceTransformer('paraphrase-multilingual-MiniLM-L12-v2')
 logger.info("Modelo SBERT cargado correctamente")
 except ImportError:
 logger.warning("No se pudo importar sentence-transformers. Desactivando
SBERT.")
 self.use_sbert = False
 except Exception as e:
 logger.warning(f"No se pudo cargar el modelo SBERT: {e}")
 self.use_sbert = False

 # Base de conocimiento ampliada para generación de prácticas
 self.base_conocimiento = [
 "Resolver ecuaciones diferenciales de primer orden utilizando métodos numéricos.",
 "Análisis de datos utilizando técnicas estadísticas descriptivas e inferenciales.",
 # ... (más ejemplos)
]

 # Entrenar el vectorizador con la base de conocimiento
 self.tfidf_matrix = self.vectorizer.fit_transform(self.base_conocimiento)

 # Cargar plantillas de retroalimentación
 self.cargar_plantillas_retroalimentacion()

 # Cargar recursos educativos por estilo de aprendizaje
 self.cargar_recursos_educativos()

 # Intentar cargar modelos pre-entrenados

```

```

self.cargar_modelos()

Diccionario para almacenar métricas de evaluación
self.metricas_evaluacion = {
 'claridad': self._evaluar_claridad,
 'profundidad': self._evaluar_profundidad,
 'estructura': self._evaluar_estructura,
 'originalidad': self._evaluar_originalidad,
 'relevancia': self._calcular_relevancia
}

Inicializar contador de evaluaciones para seguimiento
self.contador_evaluaciones = 0

Configuración de la base de datos
self.db_config = db_config
self.connection = None
if db_config:
 try:
 self.connection = mysql.connector.connect(**db_config)
 self.cursor = self.connection.cursor(dictionary=True)
 logger.info("Conexión a la base de datos establecida correctamente")
 except Error as e:
 logger.error(f"Error al conectar a la base de datos: {e}")
...

```

## Análisis detallado de la inicialización:

### 1. Configuración de stopwords:

```

```python
self.spanish_stopwords = stopwords.words('spanish')
```

```

Las stopwords son palabras comunes como "el", "la", "y" que generalmente se ignoran en el análisis de texto porque no aportan mucho significado. Aquí cargamos la lista de stopwords en español de NLTK.

### 2. Configuración del vectorizador TF-IDF:

```

```python
self.vectorizer = TfidfVectorizer(
    stop_words=self.spanish_stopwords,
    max_features=5000,
    ngram_range=(1, 3),
    min_df=2,

```

```

    max_df=0.95
)
...

```

El vectorizador TF-IDF convierte texto en vectores numéricos. Los parámetros son:

1. **stop_words**: Lista de palabras a ignorar
2. **max_features**: Limita el vocabulario a 5000 palabras más relevantes
3. **ngram_range=(1, 3)**: Considera palabras individuales, pares y tríos de palabras
4. **min_df=2**: Ignora palabras que aparecen en menos de 2 documentos
5. **max_df=0.95**: Ignora palabras que aparecen en más del 95% de los documentos

3. ****Configuración del dispositivo****:

```

```python
self.device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
```

```

Detecta si hay una GPU disponible (CUDA) y la usa si existe. Las GPUs aceleran significativamente el entrenamiento de redes neuronales.

4. ****Inicialización de modelos****:

```

```python
try:
 # Crear modelos
 self.modelo_contenido = EnhancedContenidoEvaluator(input_dim=5000,
hidden_dim=768)
 self.modelo_codigo = EnhancedCodigoEvaluator(input_dim=5000, hidden_dim=768)

 # Mover modelos al dispositivo
 self.modelo_contenido = self.modelo_contenido.to(self.device)
 self.modelo_codigo = self.modelo_codigo.to(self.device)

 # Optimizadores
 self.optimizer_contenido = optim.Adam(self.modelo_contenido.parameters(), lr=0.001,
weight_decay=1e-5)
 self.optimizer_codigo = optim.Adam(self.modelo_codigo.parameters(), lr=0.001,
weight_decay=1e-5)

 # Función de pérdida
 self.criterion = nn.MSELoss()

except Exception as e:
 logger.error(f"Error al inicializar modelos: {e}")

```

```

 self.modelo_contenido = DummyModel()
 self.modelo_codigo = DummyModel()
 ...

```

Aquí:

1. Creamos dos modelos: uno para contenido textual y otro para código
2. Los movemos al dispositivo adecuado (GPU o CPU)
3. Configuramos optimizadores Adam con:

1. **lr=0.001**: Tasa de aprendizaje (cuánto ajustar los pesos en cada paso)
2. **weight\_decay=1e-5**: Regularización para prevenir sobreajuste

4. Usamos MSE (Error Cuadrático Medio) como función de pérdida
5. Si algo falla, usamos modelos dummy como respaldo

5. **Configuración de SBERT (opcional)**:

```

``python
self.use_sbert = use_sbert
if use_sbert:
 try:
 from sentence_transformers import SentenceTransformer
 self.sbert_model = SentenceTransformer('paraphrase-multilingual-MiniLM-L12-v2')
 logger.info("Modelo SBERT cargado correctamente")
 except ImportError:
 logger.warning("No se pudo importar sentence-transformers. Desactivando SBERT.")
 self.use_sbert = False
 except Exception as e:
 logger.warning(f"No se pudo cargar el modelo SBERT: {e}")
 self.use_sbert = False
...

```

SBERT (Sentence-BERT) es un modelo avanzado para análisis semántico que captura mejor el significado de las frases. Es opcional porque:

1. Requiere más recursos
2. Necesita una biblioteca adicional (sentence-transformers)
3. No es esencial para el funcionamiento básico del sistema

6. **Base de conocimiento**:

```

'''python
self.base_conocimiento = [
 "Resolver ecuaciones diferenciales de primer orden utilizando métodos numéricos.",
 "Análisis de datos utilizando técnicas estadísticas descriptivas e inferenciales.",
 # ... (más ejemplos)
]

self.tfidf_matrix = self.vectorizer.fit_transform(self.base_conocimiento)
'''

```

Esta base de conocimiento contiene ejemplos de prácticas educativas. Se usa para:

1. Entrenar inicialmente el vectorizador
2. Generar nuevas prácticas por similitud
3. Proporcionar un punto de referencia para evaluar relevancia

#### 7. **\*\*Carga de recursos adicionales\*\***:

```

'''python
self.cargar_plantillas_retroalimentacion()
self.cargar_recursos_educativos()
self.cargar_modelos()
'''

```

Estos métodos cargan:

1. Plantillas de retroalimentación para diferentes niveles de calificación
2. Recursos educativos categorizados por estilo de aprendizaje
3. Modelos pre-entrenados si existen

#### 8. **\*\*Métricas de evaluación\*\***:

```

'''python
self.metricas_evaluacion = {
 'claridad': self._evaluar_claridad,
 'profundidad': self._evaluar_profundidad,
 'estructura': self._evaluar_estructura,
 'originalidad': self._evaluar_originalidad,
 'relevancia': self._calcular_relevancia
}
'''

```

Este diccionario mapea nombres de métricas a los métodos que las calculan, facilitando la evaluación dinámica de diferentes aspectos.

9. **\*\*Configuración de base de datos (opcional)\*\*:**

```
```python
self.db_config = db_config
self.connection = None
if db_config:
    try:
        self.connection = mysql.connector.connect(**db_config)
        self.cursor = self.connection.cursor(dictionary=True)
        logger.info("Conexión a la base de datos establecida correctamente")
    except Error as e:
        logger.error(f"Error al conectar a la base de datos: {e}")
```
```

Si se proporciona una configuración de base de datos, el sistema se conecta a MySQL para:

1. Almacenar resultados de evaluaciones
2. Recuperar datos históricos
3. Mantener registros de estudiantes

---

## ## 6. Procesamiento de Texto y Vectorización

Vamos a analizar cómo el sistema procesa y vectoriza el texto:

### ### Método de preprocesamiento

```
```python
def _preprocess_text(self, text):
    """Preprocesa el texto para análisis."""
    if not text:
        return ""

    # Convertir a minúsculas
    text = text.lower()

    # Eliminar caracteres especiales pero mantener puntuación importante
    text = re.sub(r'[^w\s.,;¿?¡!()-]', '', text)

    # Eliminar espacios extra
```

```

text = re.sub(r'\s+', ' ', text).strip()

return text
...

```

¿Por qué preprocesamos el texto?

El preprocesamiento es como "limpiar" el texto para que sea más fácil de analizar. Incluye:

1. **Convertir a minúsculas**: Para que "Python" y "python" se consideren la misma palabra.
2. **Eliminar caracteres especiales**: Manteniendo puntuación importante para el análisis.
3. **Normalizar espacios**: Eliminando espacios extra y asegurando consistencia.

Este preprocesamiento es crucial porque:

- Reduce la variabilidad innecesaria en el texto
- Mejora la precisión de la vectorización
- Facilita la comparación entre textos

Detección del tipo de contenido

```

```python
def detectar_tipo_contenido(self, contenido: str) -> str:
 """
 Detecta automáticamente el tipo de contenido para aplicar el evaluador adecuado.

```

Args:

contenido: Texto a analizar

Returns:

Tipo de contenido: 'codigo', 'texto', 'matematico', etc.

```

 """
 # Indicadores de código
 indicadores_codigo = [
 'def ', 'class ', 'function', 'import ', 'from ', 'return ', 'if ', 'else:', 'for ', 'while ',
 '{', '}', '(', ')', 'public ', 'private ', 'protected ', 'static ', 'void ', 'int ', 'float ',
 'string ', 'bool ', 'var ', 'const ', 'let ', 'console.log', 'print(', 'System.out', 'cout <<',
 'SELECT ', 'FROM ', 'WHERE ', 'INSERT INTO', 'UPDATE ', 'DELETE FROM',
 'CREATE TABLE'
]

```

# Indicadores de contenido matemático

```

indicadores_matematico = [
 '\\begin{equation}', '\\end{equation}', '\\frac', '\\sum', '\\int', '\\prod', '\\lim',
 '\\alpha', '\\beta', '\\gamma', '\\delta', '\\theta', '\\lambda', '\\pi', '\\sigma',
 '\\sqrt', '\\partial', '\\nabla', '\\infty', '\\approx', '\\neq', '\\geq', '\\leq',

```

```

 '\in', '\subset', '\cup', '\cap', '\emptyset', '\mathbb', '\mathcal', '\mathrm'
]

 # Contar indicadores
 contador_codigo = sum(1 for ind in indicadores_codigo if ind in contenido)
 contador_matematico = sum(1 for ind in indicadores_matematico if ind in contenido)

 # Calcular porcentajes (normalizado por la cantidad de indicadores)
 porcentaje_codigo = contador_codigo / len(indicadores_codigo)
 porcentaje_matematico = contador_matematico / len(indicadores_matematico)

 # Determinar tipo de contenido
 if porcentaje_codigo > 0.1: # Si más del 10% de los indicadores de código están
presentes
 return 'codigo'
 elif porcentaje_matematico > 0.1: # Si más del 10% de los indicadores matemáticos
están presentes
 return 'matematico'
 else:
 return 'texto'
...

```

### ¿Cómo detecta el sistema el tipo de contenido?

Este método es como un "detector" que identifica automáticamente si el contenido es:

- **Código de programación**: Buscando palabras clave como 'def', 'class', 'if', etc.
- **Contenido matemático**: Buscando símbolos LaTeX como '\frac', '\sum', '\alpha', etc.
- **Texto general**: Si no se detecta ninguno de los anteriores

El proceso es:

1. Definir listas de indicadores para cada tipo
2. Contar cuántos indicadores de cada tipo están presentes
3. Calcular porcentajes (normalizado por la cantidad de indicadores)
4. Determinar el tipo según umbrales (>10% para código o matemático)

Esta detección automática es crucial porque permite aplicar el evaluador adecuado sin intervención manual.

### Cálculo de relevancia

```

```python
def _calcular_relevancia(self, contenido, titulo, objetivo):
    """Calcula la relevancia semántica entre el contenido y los requisitos."""
    if not titulo and not objetivo:

```

```

        return 1.0 # Si no hay título/objetivo, asumir relevancia completa

# Combinar título y objetivo
requisitos = f"{título} {objetivo}"

# Usar modelo SBERT si está disponible
if self.use_sbert:
    try:
        # Obtener embeddings
        embedding_requisitos = self.sbert_model.encode([requisitos])[0]
        embedding_contenido = self.sbert_model.encode([contenido[:512]])[0]

        # Calcular similitud del coseno
        similarity = cosine_similarity(
            [embedding_requisitos],
            [embedding_contenido]
        )[0][0]

        return max(0.0, min(1.0, similarity)) # Asegurar rango 0-1
    except Exception as e:
        logger.warning(f"Error al calcular relevancia con SBERT: {e}")
        # Fallback a método TF-IDF

# Método TF-IDF (fallback)
try:
    # Vectorizar ambos textos
    vec_requisitos = self.vectorizer.transform([requisitos])
    vec_contenido = self.vectorizer.transform([contenido])

    # Calcular similitud del coseno
    similarity = cosine_similarity(vec_requisitos, vec_contenido)[0][0]

    return max(0.0, min(1.0, similarity)) # Asegurar rango 0-1
except Exception as e:
    logger.error(f"Error al calcular relevancia con TF-IDF: {e}")

# Método de respaldo simple basado en palabras clave
palabras_clave = set(requisitos.split())
palabras_contenido = set(contenido.split())
palabras_comunes = palabras_clave.intersection(palabras_contenido)

if len(palabras_comunes) > 0:
    return len(palabras_comunes) / len(palabras_clave)
return 0.5 # Valor por defecto
...

```

¿Cómo se calcula la relevancia?

La relevancia mide cuánto se relaciona el contenido con el tema/objetivo solicitado. Es como evaluar si un estudiante respondió la pregunta correcta. El sistema usa tres métodos, en orden de preferencia:

1. ****SBERT (si está disponible)****:

1. Convierte tanto los requisitos como el contenido en "embeddings" (vectores que capturan el significado semántico)
2. Calcula la similitud del coseno entre estos vectores
3. Ventaja: Captura mejor el significado semántico, no solo coincidencia de palabras

2. ****TF-IDF (método de respaldo)****:

1. Vectoriza ambos textos usando el vectorizador TF-IDF
2. Calcula la similitud del coseno entre estos vectores
3. Ventaja: Más simple y rápido que SBERT, pero menos preciso semánticamente

3. ****Palabras clave (último recurso)****:

1. Simplemente cuenta cuántas palabras del título/objetivo aparecen en el contenido
2. Ventaja: Extremadamente simple y siempre funciona, pero muy básico

El resultado es un valor entre 0 y 1, donde:

- 0 significa completamente irrelevante
- 1 significa perfectamente relevante

7. Análisis de Métricas

El sistema evalúa diferentes métricas según el tipo de contenido. Vamos a analizar algunas de las más importantes:

Métricas para contenido textual

Claridad

```python

```

def _evaluar_claridad(self, texto):
 """Evalúa la claridad del texto."""
 # Métricas simples de claridad
 palabras = texto.split()
 if not palabras:
 return 0.5 # Valor por defecto

 # Longitud promedio de palabras (palabras más cortas suelen ser más claras)
 longitud_promedio = sum(len(p) for p in palabras) / len(palabras)
 claridad_longitud = max(0, min(1, 2 - longitud_promedio / 10))

 # Variedad de vocabulario (menos variedad suele indicar más claridad)
 vocabulario_unico = len(set(palabras))
 ratio_vocabulario = vocabulario_unico / len(palabras)
 claridad_vocabulario = max(0, min(1, 1.5 - ratio_vocabulario))

 # Presencia de conectores lógicos (mejora la claridad)
 conectores = ['porque', 'por lo tanto', 'así', 'entonces', 'además', 'sin embargo', 'aunque',
'pero']
 conectores_presentes = sum(1 for c in conectores if c in texto.lower())
 claridad_conectores = min(1, conectores_presentes / 5)

 # Combinar métricas (ponderadas)
 claridad = (0.4 * claridad_longitud + 0.3 * claridad_vocabulario + 0.3 *
claridad_conectores)

 # Escalar a 0-10
 return claridad * 10

```

##### ¿Cómo se evalúa la claridad?

La claridad mide qué tan fácil es entender el texto. El método combina tres factores:

1. **Longitud promedio de palabras**:

1. Palabras más cortas suelen ser más claras
2. Fórmula: `max(0, min(1, 2 - longitud_promedio / 10))`
3. Esto da un valor alto para palabras cortas y bajo para palabras largas

2. **Variedad de vocabulario**:

1. Menos variedad suele indicar más claridad (repetir términos clave ayuda a la comprensión)
2. Fórmula: `max(0, min(1, 1.5 - ratio_vocabulario))`
3. Esto da un valor alto para poca variedad y bajo para mucha variedad

3. **\*\*Presencia de conectores lógicos\*\***:

1. Conectores como "porque", "por lo tanto", "sin embargo" mejoran la claridad
2. Fórmula:  $\text{min}(1, \text{conectores\_presentes} / 5)$
3. Esto da un valor alto si hay muchos conectores

Estos factores se combinan con pesos diferentes:

- 40% para longitud de palabras
- 30% para variedad de vocabulario
- 30% para conectores lógicos

El resultado se escala a un rango de 0-10.

#### Profundidad

```
```python
def _evaluar_profundidad(self, texto):
    """Evalúa la profundidad del análisis en el texto."""
    palabras = texto.split()
    if not palabras:
        return 0.5 # Valor por defecto

    # Longitud del texto (textos más largos suelen tener más profundidad)
    longitud = len(palabras)
    profundidad_longitud = min(1, longitud / 500)

    # Variedad de vocabulario (más variedad suele indicar más profundidad)
    vocabulario_unico = len(set(palabras))
    ratio_vocabulario = vocabulario_unico / len(palabras) if len(palabras) > 0 else 0
    profundidad_vocabulario = min(1, ratio_vocabulario * 3)

    # Presencia de términos técnicos o específicos
    terminos_tecnicos = ['análisis', 'metodología', 'implementación', 'algoritmo', 'estructura',
                          'sistema', 'proceso', 'función', 'componente', 'arquitectura', 'paradigma',
                          'teoría', 'concepto', 'principio', 'framework', 'modelo', 'patrón']
    terminos_presentes = sum(1 for t in terminos_tecnicos if t in texto.lower())
    profundidad_terminos = min(1, terminos_presentes / 10)

    # Combinar métricas (ponderadas)
```

```
profundidad = (0.3 * profundidad_longitud + 0.4 * profundidad_vocabulario + 0.3 *
profundidad_terminos)
```

```
# Escalar a 0-10
return profundidad * 10
'''
```

¿Cómo se evalúa la profundidad?

La profundidad mide qué tan detallado y completo es el análisis. El método combina tres factores:

1. **Longitud del texto**:

1. Textos más largos suelen tener más profundidad
2. Fórmula: ``min(1, longitud / 500)``
3. Esto da un valor alto para textos largos (>500 palabras)

2. **Variedad de vocabulario**:

1. Más variedad suele indicar más profundidad (contrario a la claridad)
2. Fórmula: ``min(1, ratio_vocabulario * 3)``
3. Esto da un valor alto para mucha variedad

3. **Presencia de términos técnicos**:

1. Términos como "análisis", "metodología", "algoritmo" indican profundidad
2. Fórmula: ``min(1, terminos_presentes / 10)``
3. Esto da un valor alto si hay muchos términos técnicos

Estos factores se combinan con pesos diferentes:

- 30% para longitud
- 40% para variedad de vocabulario
- 30% para términos técnicos

El resultado se escala a un rango de 0-10.

Métricas para código

Funcionalidad

```
```python
def _evaluar_funcionalidad_codigo(self, codigo):
 """Evalúa la funcionalidad del código."""
 # Indicadores de buena funcionalidad
 indicadores_positivos = [
 'return', 'print', 'if', 'else', 'for', 'while', 'try', 'except',
 'def', 'class', 'import', 'from'
]

 # Indicadores de problemas de funcionalidad
 indicadores_negativos = [
 'error', 'exception', 'fail', 'bug', 'issue', 'problem',
 '# TODO', '# FIXME', 'pass #'
]

 # Contar indicadores
 positivos = sum(1 for ind in indicadores_positivos if ind in codigo)
 negativos = sum(1 for ind in indicadores_negativos if ind in codigo)

 # Calcular puntuación base
 puntuacion_base = min(1, positivos / 10) * 0.8 - min(0.5, negativos / 5) * 0.2

 # Verificar estructura básica de funciones/clases
 tiene_funciones = 'def' in codigo
 tiene_clases = 'class' in codigo
 tiene_main = 'if __name__' in codigo or 'main()' in codigo

 # Bonificación por estructura adecuada
 bonificacion_estructura = 0
 if tiene_funciones:
 bonificacion_estructura += 0.1
 if tiene_clases:
 bonificacion_estructura += 0.1
 if tiene_main:
 bonificacion_estructura += 0.1

 # Calcular puntuación final
 funcionalidad = max(0, min(1, puntuacion_base + bonificacion_estructura))

 # Escalar a 0-10
 return funcionalidad * 10
```
```

¿Cómo se evalúa la funcionalidad del código?

La funcionalidad mide si el código parece estar correctamente implementado y funcional. El método combina varios factores:

1. ****Indicadores positivos****:

1. Presencia de elementos como 'return', 'if', 'def', 'class', etc.
2. Estos indican código estructurado y completo
3. Fórmula: ``min(1, positivos / 10) * 0.8``

2. ****Indicadores negativos****:

1. Presencia de palabras como 'error', 'bug', 'TODO', etc.
2. Estos indican código incompleto o problemático
3. Fórmula: ``min(0.5, negativos / 5) * 0.2`` (restado)

3. ****Estructura básica****:

1. Verifica si el código tiene funciones, clases, punto de entrada main
2. Cada elemento presente añade una bonificación de 0.1

La puntuación final se calcula como:

``puntuación_base + bonificación_estructura``

Y se limita al rango 0-1 antes de escalarse a 0-10.

Eficiencia

```
```python
def _evaluar_eficiencia_codigo(self, codigo):
 """Evalúa la eficiencia del código."""
 # Indicadores de buena eficiencia
 indicadores_positivos = [
 'O(n)', 'O(1)', 'O(log n)', 'optimiz', 'eficien',
 'dict', 'set', 'list comprehension', 'generator', 'yield',
 'cache', 'memoiz'
]

 # Indicadores de problemas de eficiencia
 indicadores_negativos = [
 'O(n^2)', 'O(n^3)', 'O(2^n)', 'for i in range', 'for j in range',

```

```

 'while True', 'sleep(', 'time.sleep'
]

Contar indicadores
positivos = sum(1 for ind in indicadores_positivos if ind in codigo)
negativos = sum(1 for ind in indicadores_negativos if ind in codigo)

Calcular puntuación base
puntuacion_base = min(1, positivos / 5) * 0.7 - min(0.5, negativos / 3) * 0.3

Verificar uso de estructuras de datos eficientes
usa_dict = 'dict(' in codigo or '{' in codigo
usa_set = 'set(' in codigo
usa_list_comp = '[' in codigo and 'for' in codigo and ']' in codigo

Bonificación por uso de estructuras eficientes
bonificacion_estructuras = 0
if usa_dict:
 bonificacion_estructuras += 0.1
if usa_set:
 bonificacion_estructuras += 0.1
if usa_list_comp:
 bonificacion_estructuras += 0.1

Calcular puntuación final
eficiencia = max(0, min(1, puntuacion_base + bonificacion_estructuras))

Escalar a 0-10
return eficiencia * 10
'''

```

##### ¿Cómo se evalúa la eficiencia del código?

La eficiencia mide si el código utiliza algoritmos y estructuras de datos óptimos. El método combina:

1. **\*\*Indicadores positivos\*\***:

1. Presencia de notaciones de complejidad eficientes ('O(n)', 'O(1)', 'O(log n)')
2. Uso de términos relacionados con optimización
3. Estructuras de datos eficientes (dict, set, list comprehension)
4. Fórmula: ``min(1, positivos / 5) * 0.7``

2. **\*\*Indicadores negativos\*\***:

1. Notaciones de complejidad ineficientes ('O(n^2)', 'O(n^3)', 'O(2^n)')

2. Bucles anidados o potencialmente infinitos
3. Operaciones que bloquean la ejecución (sleep)
4. Fórmula: ``min(0.5, negativos / 3) * 0.3`` (restado)

### 3. **\*\*Uso de estructuras eficientes\*\***:

1. Verifica el uso de diccionarios, conjuntos y comprensiones de lista
2. Cada estructura presente añade una bonificación de 0.1

La puntuación final se calcula como:

``puntuación_base + bonificación_estructuras``

Y se limita al rango 0-1 antes de escalarse a 0-10.

---

## ## 8. Generación de Retroalimentación

El sistema genera retroalimentación detallada basada en la evaluación. Vamos a analizar cómo funciona:

### ### Carga de plantillas de retroalimentación

```
```python
def cargar_plantillas_retroalimentacion(self):
    """Carga plantillas de retroalimentación más detalladas y variadas"""
    self.feedback_templates = {
        'excellent': [
            "Excelente trabajo. Has demostrado un dominio completo del tema, con un análisis profundo y bien estructurado.",
            "Trabajo sobresaliente. Cumples con todos los requisitos y vas más allá, demostrando comprensión avanzada de los conceptos.",
            # ... más plantillas
        ],
        'very_good': [
            "Muy buen trabajo. Demuestras una sólida comprensión del tema con un análisis bien desarrollado.",
            # ... más plantillas
        ],
        # ... más categorías
    }
```

```

# Plantillas específicas para código
self.code_feedback_templates = {
    'excellent': [
        "Excelente implementación. Tu código es eficiente, bien estructurado y sigue las
mejores prácticas de programación.",
        # ... más plantillas
    ],
    # ... más categorías
}

# Plantillas específicas para diferentes tipos de errores
self.error_templates = {
    'conceptual': [
        "Hay un error conceptual importante en tu explicación de {concepto}.
{explicacion_correcta}",
        # ... más plantillas
    ],
    # ... más categorías
}

# ... más tipos de plantillas
...

```

¿Por qué usar plantillas de retroalimentación?

Las plantillas proporcionan una base consistente para la retroalimentación, que luego se personaliza según:

- La calificación obtenida
- Las fortalezas y debilidades identificadas
- El estilo de aprendizaje del estudiante

Esto es similar a cómo un profesor tiene "frases estándar" que adapta a cada estudiante.

Identificación de fortalezas y debilidades

```

```python
def _identificar_fortalezas_debilidades(self, contenido, metricas, calificacion):
 """Identifica fortalezas y debilidades en el contenido textual."""
 fortalezas = []
 debilidades = []

 # Analizar métricas para identificar fortalezas
 if metricas['claridad'] >= 7.5:
 fortalezas.append("Excelente claridad en la exposición de ideas")
 elif metricas['claridad'] >= 6.0:
 fortalezas.append("Buena claridad en la presentación del contenido")

```

```

... más análisis de métricas

Análisis adicional del contenido
palabras = contenido.split()

Verificar longitud
if len(palabras) < 200:
 debilidades.append("El trabajo es demasiado breve para desarrollar adecuadamente el tema")
elif len(palabras) > 2000:
 fortalezas.append("Trabajo extenso con desarrollo detallado")

Verificar variedad de vocabulario
vocabulario_unico = len(set(palabras))
ratio_vocabulario = vocabulario_unico / len(palabras) if len(palabras) > 0 else 0
if ratio_vocabulario > 0.5:
 fortalezas.append("Excelente riqueza de vocabulario")
elif ratio_vocabulario < 0.3:
 debilidades.append("Vocabulario limitado y repetitivo")

return fortalezas, debilidades
...

```

#### ¿Cómo identifica el sistema fortalezas y debilidades?

El sistema analiza:

1. **Métricas calculadas**:

1. Compara cada métrica con umbrales (excelente:  $\geq 7.5$ , bueno:  $\geq 6.0$ , etc.)
2. Asigna descripciones específicas según el nivel

2. **Características adicionales**:

1. Longitud del texto (demasiado breve, extenso, etc.)
2. Variedad de vocabulario (rico, limitado, etc.)
3. Para código: uso de funciones/clases, modularización, etc.

Este análisis es crucial para proporcionar retroalimentación específica y útil.

#### Generación de comentarios detallados

```

python
def _generar_comentarios_detallados(self, calificacion, contenido, titulo, objetivo, fortalezas,
debilidades, estilo_aprendizaje):
 """Genera comentarios detallados para contenido textual."""
 # Determinar categoría de calificación
 if calificacion >= 9.0:
 categoria = 'excellent'
 elif calificacion >= 8.0:
 categoria = 'very_good'
 elif calificacion >= 7.0:
 categoria = 'good'
 elif calificacion >= 5.0:
 categoria = 'average'
 elif calificacion >= 3.0:
 categoria = 'needs_improvement'
 else:
 categoria = 'poor'

 # Seleccionar plantilla base según categoría
 import random
 plantilla_base = random.choice(self.feedback_templates[categoria])

 # Personalizar comentario según estilo de aprendizaje
 comentario_personalizado = ""
 if estilo_aprendizaje:
 estilos = [e.strip().lower() for e in estilo_aprendizaje.split(',')]

 if 'visual' in estilos:
 comentario_personalizado += "Te recomiendo organizar tus ideas en mapas
conceptuales o diagramas para reforzar tu comprensión visual del tema."

 # ... más estilos

 # Incorporar fortalezas y debilidades
 comentario_fortalezas = ""
 if fortalezas:
 comentario_fortalezas = "\n\nFortalezas destacadas:\n- " + "\n- ".join(fortalezas[:3])

 comentario_debilidades = ""
 if debilidades:
 comentario_debilidades = "\n\nAspectos a mejorar:\n- " + "\n- ".join(debilidades[:3])

 # Construir comentario completo
 comentario_completo = f"{plantilla_base}\n\n"

 if comentario_personalizado:

```

```
comentario_completo += f"Considerando tu estilo de aprendizaje:
{comentario_personalizado}\n"
```

```
comentario_completo += f"{comentario_fortalezas}{comentario_debilidades}"
```

```
return comentario_completo
...
```

### ¿Cómo genera el sistema comentarios detallados?

El proceso tiene varios pasos:

1. **Determinar categoría** según la calificación:

1. excellent (9.0-10)
2. very\_good (8.0-8.9)
3. good (7.0-7.9)
4. average (5.0-6.9)
5. needs\_improvement (3.0-4.9)
6. poor (0-2.9)

2. **Seleccionar plantilla base** aleatoriamente de la categoría correspondiente

3. **Personalizar según estilo de aprendizaje**:

1. Visual: Recomienda mapas conceptuales, diagramas
2. Auditivo: Recomienda explicar ideas en voz alta, discutir
3. Kinestésico: Recomienda aplicar conceptos en ejercicios prácticos
4. Lectura/Escritura: Recomienda desarrollar ideas por escrito

4. **Incorporar fortalezas y debilidades** identificadas

5. **Construir comentario completo** combinando todos los elementos

Este enfoque proporciona retroalimentación personalizada y específica, similar a lo que haría un profesor dedicado.

### Generación de sugerencias de mejora

```
```python  
def _generar_sugerencias_mejora(self, calificacion, contenido, titulo, objetivo, debilidades,  
estilo_aprendizaje):  
    """Genera sugerencias específicas para mejorar el contenido textual."""  
    sugerencias = []
```

```

# Sugerencias basadas en debilidades
for debilidad in debilidades[:3]: # Limitar a las 3 principales debilidades
    if "claridad" in debilidad.lower():
        sugerencias.append("Mejora la claridad utilizando ejemplos concretos y
explicaciones más detalladas.")
    elif "profundidad" in debilidad.lower() or "superficial" in debilidad.lower():
        sugerencias.append("Profundiza en el análisis investigando más fuentes y
desarrollando argumentos más complejos.")
    # ... más casos

# Sugerencias generales según calificación
if calificacion < 5.0:
    sugerencias.append("Revisa los conceptos fundamentales del tema y asegúrate de
comprenderlos correctamente.")
    sugerencias.append("Considera reescribir completamente el trabajo siguiendo una
estructura más clara.")
elif calificacion < 7.0:
    sugerencias.append("Revisa la coherencia entre párrafos para asegurar una
progresión lógica de ideas.")
    sugerencias.append("Incluye más referencias o ejemplos para respaldar tus
argumentos.")
else:
    sugerencias.append("Para mejorar aún más, considera incorporar perspectivas críticas
o comparativas.")
    sugerencias.append("Refuerza tus conclusiones conectándolas claramente con los
puntos principales desarrollados.")

# Sugerencias personalizadas según estilo de aprendizaje
if estilo_aprendizaje:
    estilos = [e.strip().lower() for e in estilo_aprendizaje.split(',')]

    if 'visual' in estilos:
        sugerencias.append("Intenta crear mapas conceptuales o diagramas antes de
escribir para organizar mejor tus ideas.")
    # ... más estilos

# Formatear sugerencias
sugerencias_texto = "Para mejorar tu trabajo:\n\n- " + "\n- ".join(sugerencias[:5]) # Limitar
a 5 sugerencias

return sugerencias_texto
...

```

¿Cómo genera el sistema sugerencias de mejora?

El sistema genera sugerencias de tres fuentes:

1. **Basadas en debilidades detectadas**:

1. Para cada debilidad, proporciona una sugerencia específica
2. Por ejemplo, si hay problemas de claridad, sugiere usar ejemplos concretos

2. ****Según nivel de calificación****:

1. Para calificaciones bajas (<5.0): Sugerencias fundamentales
2. Para calificaciones medias (<7.0): Sugerencias de mejora
3. Para calificaciones altas (≥7.0): Sugerencias de perfeccionamiento

3. ****Personalizadas por estilo de aprendizaje****:

1. Visual: Sugiere usar mapas conceptuales
2. Auditivo: Sugiere grabar ideas en audio
3. Kinestésico: Sugiere relacionar conceptos con experiencias prácticas
4. Lectura/Escritura: Sugiere practicar escribiendo resúmenes

Las sugerencias se limitan a 5 para no abrumar al estudiante.

Recomendación de recursos educativos

```
``python
def _recomendar_recursos(self, estilo_aprendizaje, titulo, objetivo, calificacion):
    """Recomienda recursos educativos según el estilo de aprendizaje y tema."""
    recursos = []

    # Determinar categoría temática
    categoria = self._determinar_categoria_tematica(titulo, objetivo)

    # Si no hay estilo de aprendizaje definido, recomendar recursos variados
    if not estilo_aprendizaje:
        # Seleccionar recursos variados
        for estilo in ['visual', 'auditivo', 'kinestesico', 'lectura_escritura']:
            if categoria in self.recursos_educativos[estilo]:
                # Seleccionar un recurso aleatorio de cada estilo
                import random
                recurso = random.choice(self.recursos_educativos[estilo][categoria])
                recursos.append(recurso)
    else:
        # Seleccionar recursos según estilos de aprendizaje del estudiante
```

```

estilos = [e.strip().lower() for e in estilo_aprendizaje.split(',')]

for estilo in estilos:
    if estilo in self.recursos_educativos and categoria in self.recursos_educativos[estilo]:
        # Seleccionar hasta 2 recursos por estilo
        import random
        recursos_estilo = self.recursos_educativos[estilo][categoria]
        num_recursos = min(2, len(recursos_estilo))
        seleccion = random.sample(recursos_estilo, num_recursos)
        recursos.extend(seleccion)

# Si no se encontraron recursos específicos, recomendar recursos generales
if not recursos:
    recursos_generales = [
        {"titulo": "Khan Academy - Cursos gratuitos en línea", "url":
"https://es.khanacademy.org/"},
        # ... más recursos
    ]
    import random
    recursos = random.sample(recursos_generales, min(3, len(recursos_generales)))

# Limitar a máximo 5 recursos
return recursos[:5]
...

```

¿Cómo recomienda el sistema recursos educativos?

El sistema sigue estos pasos:

1. ****Determina la categoría temática**** (programación, matemáticas, ciencias) basada en el título y objetivo
2. ****Selecciona recursos según el estilo de aprendizaje****:
 1. Si no hay estilo definido: selecciona recursos variados de cada estilo
 2. Si hay estilos definidos: selecciona recursos específicos para esos estilos
3. ****Si no encuentra recursos específicos****, recomienda recursos generales como Khan Academy
4. ****Limita a 5 recursos**** para no abrumar al estudiante

Los recursos están organizados en una estructura de datos por:

- Estilo de aprendizaje (visual, auditivo, kinestésico, lectura/escritura)
- Categoría temática (programación, matemáticas, ciencias)

Cada recurso incluye título y URL.

9. Entrenamiento del Modelo

El sistema incluye capacidades para entrenar los modelos con nuevos datos:

```
```python
def entrenar_modelo(self, entradas, salidas, epochs=50, batch_size=8, validation_split=0.2):
 """
```

Entrena el modelo con nuevos datos.

Args:

entradas: Lista de textos de entrada

salidas: Lista de calificaciones (0-10)

epochs: Número de épocas de entrenamiento

batch\_size: Tamaño del lote para entrenamiento

validation\_split: Proporción de datos para validación

Returns:

Diccionario con resultados del entrenamiento

```
"""
```

```
if len(entradas) != len(salidas):
```

```
 raise ValueError("Las listas de entradas y salidas deben tener la misma longitud")
```

```
if len(entradas) < 5:
```

```
 raise ValueError("Se necesitan al menos 5 ejemplos para entrenar el modelo")
```

```
try:
```

```
 # Preprocesar textos
```

```
 textos_procesados = [self._preprocess_text(texto) for texto in entradas]
```

```
 # Actualizar vectorizador
```

```
 self.vectorizer.fit(textos_procesados)
```

```
 # Crear conjuntos de datos
```

```
 dataset = TextDataset(textos_procesados, salidas, self.vectorizer)
```

```
 # Dividir en entrenamiento y validación
```

```
 from torch.utils.data import random_split
```

```
 val_size = max(1, int(len(dataset) * validation_split))
```

```
 train_size = len(dataset) - val_size
```

```
 train_dataset, val_dataset = random_split(dataset, [train_size, val_size])
```

```
 # Crear dataloaders
```

```
 from torch.utils.data import DataLoader
```

```
train_loader = DataLoader(train_dataset, batch_size=batch_size, shuffle=True)
val_loader = DataLoader(val_dataset, batch_size=batch_size)
```

```
Entrenar modelo de contenido
self.modelo_contenido.train()
optimizer = self.optimizer_contenido
```

```
Inicializar variables de seguimiento
mejor_val_loss = float('inf')
epocas_sin_mejora = 0
mejor_modelo_state = None
```

```
Historial de entrenamiento
historial = {
 'train_loss': [],
 'val_loss': []
}
```

```
Bucle de entrenamiento
for epoch in range(epochs):
 # Entrenamiento
 self.modelo_contenido.train()
 train_loss = 0.0
 for inputs, targets in train_loader:
 inputs, targets = inputs.to(self.device), targets.to(self.device)
```

```
 # Forward pass
 outputs = self.modelo_contenido(inputs)
 loss = self.criterion(outputs['global'].squeeze(), targets)
```

```
 # Backward y optimización
 optimizer.zero_grad()
 loss.backward()
 optimizer.step()
```

```
 train_loss += loss.item()
```

```
train_loss /= len(train_loader)
historial['train_loss'].append(train_loss)
```

```
Validación
self.modelo_contenido.eval()
val_loss = 0.0
with torch.no_grad():
 for inputs, targets in val_loader:
 inputs, targets = inputs.to(self.device), targets.to(self.device)
 outputs = self.modelo_contenido(inputs)
 loss = self.criterion(outputs['global'].squeeze(), targets)
```

```

 val_loss += loss.item()

 val_loss /= len(val_loader)
 historial['val_loss'].append(val_loss)

 # Guardar mejor modelo
 if val_loss < mejor_val_loss:
 mejor_val_loss = val_loss
 mejor_modelo_state = self.modelo_contenido.state_dict().copy()
 epocas_sin_mejora = 0
 else:
 epocas_sin_mejora += 1

 # Early stopping
 if epocas_sin_mejora >= 10:
 logger.info(f"Early stopping en época {epoch+1}")
 break

 logger.info(f"Época {epoch+1}/{epochs}, Loss: {train_loss:.4f}, Val Loss:
{val_loss:.4f}")

 # Cargar el mejor modelo
 if mejor_modelo_state:
 self.modelo_contenido.load_state_dict(mejor_modelo_state)

 # ... (código similar para entrenar modelo_codigo)

 # Guardar modelos
 self.guardar_modelos()

 return {
 'status': 'success',
 'epochs_completed': epoch + 1,
 'train_loss': train_loss,
 'val_loss': val_loss,
 'best_val_loss': mejor_val_loss,
 'historial': historial
 }

except Exception as e:
 logger.error(f"Error en entrenamiento: {e}")
 import traceback
 logger.error(traceback.format_exc())
 return {
 'status': 'error',
 'message': str(e)
 }
...

```

### ### ¿Cómo funciona el entrenamiento del modelo?

El entrenamiento es el proceso por el cual el modelo "aprende" a partir de ejemplos. Sigue estos pasos:

#### 1. **\*\*Preparación de datos\*\***:

1. Preprocesa los textos
2. Actualiza el vectorizador con los nuevos datos
3. Crea un dataset con TextDataset
4. Divide los datos en conjuntos de entrenamiento y validación

#### 2. **\*\*Configuración del entrenamiento\*\***:

1. Crea DataLoaders para procesar datos por lotes
2. Configura el modelo en modo entrenamiento
3. Inicializa variables de seguimiento

#### 3. **\*\*Bucle de entrenamiento\*\*** (para cada época):

##### 1. **\*\*Fase de entrenamiento\*\***:

1. Pasa los datos por el modelo (forward pass)
2. Calcula la pérdida (diferencia entre predicción y valor real)
3. Calcula gradientes (backward pass)
4. Actualiza los pesos del modelo (optimizer.step())

##### 2. **\*\*Fase de validación\*\***:

1. Evalúa el modelo en datos no vistos durante el entrenamiento
2. Calcula la pérdida de validación

##### 3. **\*\*Seguimiento del mejor modelo\*\***:

1. Si la pérdida de validación mejora, guarda el estado del modelo
2. Si no mejora durante 10 épocas, detiene el entrenamiento (early stopping)

#### 4. **\*\*Finalización\*\***:

1. Carga el mejor modelo encontrado
2. Guarda los modelos en disco
3. Retorna estadísticas del entrenamiento

Este proceso es similar a cómo un profesor mejora sus criterios de evaluación con la experiencia.

#### ### Persistencia de modelos

```
```python
def guardar_modelos(self, ruta_base='modelos'):
    """Guarda los modelos y vectorizador en disco."""
    try:
        # Crear directorio si no existe
        os.makedirs(ruta_base, exist_ok=True)

        # Guardar modelos
        torch.save(self.modelo_contenido.state_dict(), os.path.join(ruta_base,
'modelo_contenido_enhanced.pt'))
        torch.save(self.modelo_codigo.state_dict(), os.path.join(ruta_base,
'modelo_codigo_enhanced.pt'))

        # Guardar vectorizador
        with open(os.path.join(ruta_base, 'vectorizer_enhanced.pkl'), 'wb') as f:
            pickle.dump(self.vectorizer, f)

        logger.info(f"Modelos guardados en {ruta_base}")
        return True
    except Exception as e:
        logger.error(f"Error al guardar modelos: {e}")
        return False

def cargar_modelos(self, ruta_base='modelos'):
    """Carga los modelos y vectorizador desde disco."""
    try:
        # Verificar si los archivos existen
        ruta_contenido = os.path.join(ruta_base, 'modelo_contenido_enhanced.pt')
        ruta_codigo = os.path.join(ruta_base, 'modelo_codigo_enhanced.pt')
        ruta_vectorizer = os.path.join(ruta_base, 'vectorizer_enhanced.pkl')

```

```

        if not os.path.exists(ruta_contenido) or not os.path.exists(ruta_codigo) or not
os.path.exists(ruta_vectorizer):
            # Intentar cargar modelos antiguos
            ruta_contenido_old = os.path.join(ruta_base, 'modelo_contenido.pt')
            ruta_codigo_old = os.path.join(ruta_base, 'modelo_codigo.pt')
            ruta_vectorizer_old = os.path.join(ruta_base, 'vectorizer.pkl')

            if os.path.exists(ruta_contenido_old) and os.path.exists(ruta_codigo_old) and
os.path.exists(ruta_vectorizer_old):
                logger.info("Cargando modelos antiguos y actualizándolos...")
                # Cargar vectorizador antiguo
                with open(ruta_vectorizer_old, 'rb') as f:
                    self.vectorizer = pickle.load(f)

                # Guardar modelos nuevos (inicializados aleatoriamente)
                self.guardar_modelos(ruta_base)
                return True
            else:
                logger.warning("Archivos de modelo no encontrados. Se usarán modelos
nuevos.")
                return False

        # Cargar modelos
        self.modelo_contenido.load_state_dict(torch.load(ruta_contenido,
map_location=self.device))
        self.modelo_contenido.eval()

        self.modelo_codigo.load_state_dict(torch.load(ruta_codigo, map_location=self.device))
        self.modelo_codigo.eval()

        # Cargar vectorizador
        with open(ruta_vectorizer, 'rb') as f:
            self.vectorizer = pickle.load(f)

        logger.info(f"Modelos cargados desde {ruta_base}")
        return True
    except Exception as e:
        logger.error(f"Error al cargar modelos: {e}")
        return False
...

```

¿Por qué guardar y cargar modelos?

La persistencia de modelos es crucial porque:

1. ****El entrenamiento es costoso**** en tiempo y recursos computacionales
2. ****Permite reutilizar modelos**** sin tener que entrenarlos de nuevo

3. ****Facilita la distribución**** del sistema a otros usuarios

El sistema guarda tres componentes principales:

- ****modelo_contenido****: Para evaluar contenido textual
- ****modelo_codigo****: Para evaluar código
- ****vectorizador****: Para convertir texto en vectores

También incluye manejo de compatibilidad con versiones anteriores de los modelos.

10. Aplicación Práctica

Veamos cómo se utiliza el sistema para evaluar contenido:

```
```python
def analizar_contenido(self, contenido_archivo, titulo_actividad="", objetivo_actividad="",
estilo_aprendizaje=None):
 """
 Analiza el contenido del archivo y devuelve una calificación y comentarios detallados.
 Incorpora análisis semántico profundo, evaluación multidimensional y personalización por
 estilo de aprendizaje.
 """
```

Args:

contenido\_archivo: Texto del contenido entregado  
titulo\_actividad: Título de la actividad  
objetivo\_actividad: Objetivo de la actividad  
estilo\_aprendizaje: Estilo(s) de aprendizaje del estudiante (separados por comas)

Returns:

Diccionario con calificación, comentarios, sugerencias, relevancia y recursos recomendados

```
 """
 # Incrementar contador de evaluaciones
 self.contador_evaluaciones += 1
 logger.info(f"Iniciando análisis de contenido #{self.contador_evaluaciones}")

 # Validar que el contenido no esté vacío
 if not contenido_archivo or not contenido_archivo.strip():
 logger.warning("Contenido vacío detectado")
 return {
 "calificacion": 0.0,
 "comentarios": "El archivo está vacío o no contiene texto válido.",
 "sugerencias": "Es necesario entregar un trabajo con contenido para poder
evaluarlo.",
```

```

 "relevancia": 0.0,
 "recursos_recomendados": []
 }

 # Detectar tipo de contenido
 tipo_contenido = self.detectar_tipo_contenido(contenido_archivo)
 logger.info(f"Tipo de contenido detectado: {tipo_contenido}")

 # Preprocesar el contenido
 contenido_procesado = self._preprocess_text(contenido_archivo)
 logger.info(f"Contenido preprocesado: {len(contenido_procesado)} caracteres")

 # Análisis de relevancia
 relevancia = self._calcular_relevancia(contenido_procesado, titulo_actividad,
 objetivo_actividad)
 logger.info(f"Relevancia calculada: {relevancia:.4f}")

 # Si la relevancia es extremadamente baja, asignar calificación cero
 if relevancia < 0.2:
 logger.warning(f"Contenido irrelevante detectado (relevancia: {relevancia:.4f})")
 return self._generar_respuesta_irrelevante(estilo_aprendizaje, titulo_actividad,
 objetivo_actividad)

 # Análisis multidimensional según tipo de contenido
 if tipo_contenido == 'codigo':
 metricas = self._analizar_metricas_codigo(contenido_procesado, titulo_actividad,
 objetivo_actividad)
 else:
 metricas = self._analizar_metricas(contenido_procesado, titulo_actividad,
 objetivo_actividad)

 logger.info(f"Métricas calculadas: {metricas}")

 # Vectorizar el contenido para el modelo de evaluación
 try:
 contenido_vectorizado = self.vectorizer.transform([contenido_procesado]).toarray()[0]

 # Asegurar dimensiones consistentes
 if len(contenido_vectorizado) < 5000:
 contenido_vectorizado = np.pad(contenido_vectorizado, (0, 5000 -
len(contenido_vectorizado)), 'constant')
 elif len(contenido_vectorizado) > 5000:
 contenido_vectorizado = contenido_vectorizado[:5000]

 # Convertir a tensor
 contenido_tensor = torch.tensor(contenido_vectorizado,
dtype=torch.float32).unsqueeze(0).to(self.device)

```

```

Generar predicción base según tipo de contenido
if tipo_contenido == 'codigo':
 self.modelo_codigo.eval()
 with torch.no_grad():
 outputs = self.modelo_codigo(contenido_tensor)
 pred_base = outputs['global'].cpu().numpy()[0][0]
 logger.info(f"Predicción base del modelo de código: {pred_base}")
else:
 self.modelo_contenido.eval()
 with torch.no_grad():
 outputs = self.modelo_contenido(contenido_tensor)
 pred_base = outputs['global'].cpu().numpy()[0][0]
 logger.info(f"Predicción base del modelo de contenido: {pred_base}")
except Exception as e:
 logger.error(f"Error en la vectorización o predicción: {e}")
Usar un enfoque alternativo basado en métricas si falla la predicción
pred_base = self._calcular_calificacion_alternativa(metricas)
logger.info(f"Usando calificación alternativa: {pred_base}")

Ajustar calificación según métricas y relevancia
if tipo_contenido == 'codigo':
 calificacion_ajustada = self._ajustar_calificacion_codigo(
 pred_base,
 relevancia,
 metricas['funcionalidad'],
 metricas['eficiencia'],
 metricas['estilo'],
 metricas['documentacion']
)
else:
 calificacion_ajustada = self._ajustar_calificacion(
 pred_base,
 relevancia,
 metricas['claridad'],
 metricas['profundidad'],
 metricas['estructura']
)

Limitar a rango 0-10
calificacion_final = min(10.0, max(0.0, calificacion_ajustada))
logger.info(f"Calificación final: {calificacion_final:.2f}")

Identificar fortalezas y debilidades según tipo de contenido
if tipo_contenido == 'codigo':
 fortalezas, debilidades = self._identificar_fortalezas_debilidades_codigo(
 contenido_procesado,
 metricas,
 calificacion_final
)

```

```

)
else:
 fortalezas, debilidades = self._identificar_fortalezas_debilidades(
 contenido_procesado,
 metricas,
 calificacion_final
)

```

```

logger.info(f"Fortalezas identificadas: {len(fortalezas)}")
logger.info(f"Debilidades identificadas: {len(debilidades)}")

```

# Generar comentarios detallados según tipo de contenido

if tipo\_contenido == 'codigo':

```

 comentarios = self._generar_comentarios_detallados_codigo(
 calificacion_final,
 contenido_procesado,
 titulo_actividad,
 objetivo_actividad,
 fortalezas,
 debilidades,
 estilo_aprendizaje
)

```

else:

```

 comentarios = self._generar_comentarios_detallados(
 calificacion_final,
 contenido_procesado,
 titulo_actividad,
 objetivo_actividad,
 fortalezas,
 debilidades,
 estilo_aprendizaje
)

```

# Generar sugerencias de mejora según tipo de contenido

if tipo\_contenido == 'codigo':

```

 sugerencias = self._generar_sugerencias_mejora_codigo(
 calificacion_final,
 contenido_procesado,
 titulo_actividad,
 objetivo_actividad,
 debilidades,
 estilo_aprendizaje
)

```

else:

```

 sugerencias = self._generar_sugerencias_mejora(
 calificacion_final,
 contenido_procesado,
 titulo_actividad,

```

```

 objetivo_actividad,
 debilidades,
 estilo_aprendizaje
)

 # Recomendar recursos educativos según estilo de aprendizaje
 recursos_recomendados = self._recomendar_recursos(
 estilo_aprendizaje,
 titulo_actividad,
 objetivo_actividad,
 calificacion_final
)

 # Construir respuesta completa
 respuesta = {
 "calificacion": float(round(calificacion_final, 1)), # Asegurar que sea float
 "comentarios": comentarios,
 "sugerencias": sugerencias,
 "relevancia": float(round(relevancia * 100, 1)), # Como porcentaje
 "tipo_contenido": tipo_contenido,
 "metricas": self._formatear_metricas(metricas, tipo_contenido),
 "fortalezas": fortalezas[:3], # Limitar a 3 fortalezas principales
 "debilidades": debilidades[:3], # Limitar a 3 debilidades principales
 "recursos_recomendados": recursos_recomendados
 }

 logger.info(f"Análisis de contenido #{self.contador_evaluaciones} completado")
 return respuesta
'''

```

### ¿Cómo funciona el análisis de contenido?

Este método es el "corazón" del sistema, integrando todos los componentes para evaluar contenido. Sigue estos pasos:

1. **Validación inicial**:

1. Verifica que el contenido no esté vacío
2. Incrementa contador de evaluaciones para seguimiento

2. **Detección y preprocesamiento**:

1. Detecta automáticamente el tipo de contenido (código, texto, matemático)
2. Preprocesa el contenido para análisis

3. **\*\*Análisis de relevancia\*\***:

1. Calcula la relevancia respecto al título/objetivo
2. Si es extremadamente baja ( $<0.2$ ), genera respuesta de irrelevancia

4. **\*\*Análisis de métricas\*\***:

1. Calcula métricas específicas según el tipo de contenido
2. Para texto: claridad, profundidad, estructura, etc.
3. Para código: funcionalidad, eficiencia, estilo, etc.

5. **\*\*Predicción con modelo neural\*\***:

1. Vectoriza el contenido
2. Pasa el vector al modelo adecuado (contenido o código)
3. Obtiene predicción base
4. Si falla, usa método alternativo basado en métricas

6. **\*\*Ajuste de calificación\*\***:

1. Ajusta la predicción base según métricas y relevancia
2. Limita al rango 0-10

7. **\*\*Generación de retroalimentación\*\***:

1. Identifica fortalezas y debilidades
2. Genera comentarios detallados
3. Genera sugerencias de mejora
4. Recomienda recursos educativos

8. **\*\*Construcción de respuesta\*\***:

1. Integra todos los elementos en un diccionario
2. Incluye calificación, comentarios, sugerencias, etc.

Este proceso completo simula cómo un profesor evaluaría un trabajo, considerando múltiples aspectos y proporcionando retroalimentación detallada.

---

## ## 11. Conclusiones

### ### Resumen del sistema

El sistema de evaluación inteligente que hemos analizado es una herramienta sofisticada que combina:

1. **Procesamiento de lenguaje natural** para analizar texto y código
2. **Redes neuronales profundas** para predecir calificaciones
3. **Análisis multidimensional** de diferentes aspectos del contenido
4. **Retroalimentación personalizada** según estilo de aprendizaje
5. **Recomendación de recursos** educativos relevantes

### ### Aplicaciones prácticas

Este sistema puede utilizarse en diversos contextos educativos:

1. **Evaluación automática** de trabajos escritos y código
2. **Apoyo a profesores** para proporcionar retroalimentación detallada
3. **Autoevaluación** para estudiantes que quieren mejorar sus trabajos
4. **Generación de prácticas** educativas personalizadas
5. **Análisis de tendencias** en el desempeño de estudiantes

### ### Limitaciones y consideraciones

Es importante reconocer algunas limitaciones:

1. **Evaluación superficial**: El sistema analiza características del texto/código, pero no puede ejecutar código ni verificar la veracidad del contenido.
2. **Dependencia de datos**: La calidad de las predicciones depende de los datos de entrenamiento.
3. **Comprensión semántica limitada**: Sin SBERT, el análisis semántico es básico.
4. **Complemento, no reemplazo**: El sistema debe verse como un apoyo al criterio humano, no un reemplazo.

### ### Posibles mejoras

El sistema podría mejorarse con:

1. **Ejecución de código**: Para verificar funcionalidad real
2. **Verificación de hechos**: Para comprobar la veracidad del contenido
3. **Modelos de lenguaje más avanzados**: Como GPT o BERT para mejor comprensión semántica
4. **Interfaz de usuario**: Para facilitar su uso por profesores y estudiantes
5. **Más datos de entrenamiento**: Para mejorar la precisión de las predicciones

### ### Reflexión final

Este sistema representa un ejemplo fascinante de cómo la inteligencia artificial puede aplicarse en educación. Combina técnicas avanzadas de aprendizaje automático con conocimientos pedagógicos para crear una herramienta que potencialmente puede mejorar la experiencia educativa tanto para profesores como para estudiantes.

La clave está en utilizarlo como un complemento al juicio humano, aprovechando su capacidad para procesar grandes cantidades de contenido y proporcionar retroalimentación detallada, mientras se reconocen sus limitaciones en cuanto a comprensión profunda y contextual.